

Code-based Public Key Encryption

DSC 4110/6111: Modern Cryptography and AI/ML Security

Dr. Laltu Sardar

School of Data Science,
Indian Institute of Science Education and Research Thiruvananthapuram (IISER TVM)



September 11 & 12, 2026

Table of Content

- 1 Cryptographically Secure Channel
- 2 Revisiting PKI
- 3 Coding Theory
- 4 Error Correcting code
- 5 Error Correcting Code to PKE
- 6 Reason behind studying Code-based PKE

Cryptographically Secure Channel

September 11, 2026

Cryptographically Secure Channel

Question

How can two parties communicate securely over an insecure network?

A **cryptographically secure channel** allows two parties to communicate while providing confidentiality, integrity, and authentication.

Secure Channel = Authentication + Shared Secret + Symmetric Encryption

This is mainly achieved using a **Public Key Infrastructure (PKI)**:

Digital Signature → Authenticate Public Key → Establish Shared Key → Symmetric Encryption

What have we already discussed?

- **Signature algorithms:** Hash-based signatures for authenticating public keys.
- **Symmetric cryptography:** Efficient encryption of long messages using session keys.

We assume both components provide the required **post-quantum security**.

What remains? The main remaining problem is: **How do two parties establish a shared secret?**

Revisiting Public key infrastructure (PKI)

Public-Key Encryption: The Big Picture

Question

Suppose two users, Alice and Bob, want to communicate securely over an insecure network. What do they need?

Public-Key Encryption: The Big Picture

Question

Suppose two users, Alice and Bob, want to communicate securely over an insecure network. What do they need?

We need three major components:

- 1 **Authentication:** How does Alice know that a public key really belongs to Bob?
- 2 **Key establishment:** How can Alice and Bob establish a common secret?
- 3 **Symmetric encryption:** How do they use the common secret to protect their communication?

Authentication + Key Establishment + Symmetric Encryption

Step 1: Public-Key Infrastructure

Question

Alice receives a public key pk_B claiming to belong to Bob. How does Alice know that this key actually belongs to Bob?

Step 1: Public-Key Infrastructure

Question

Alice receives a public key pk_B claiming to belong to Bob. How does Alice know that this key actually belongs to Bob?

Each user u generates a key pair:

$$(sk_u, pk_u) \leftarrow \text{KeyGen}(1^\lambda).$$

The public key is certified by a **Certificate Authority (CA)** using a digital signature:

$$\text{Cert}_u = \text{Sign}_{sk_{CA}}(u, pk_u, \text{metadata}).$$

Alice verifies the certificate using the CA's public verification key.

CA signature $\Rightarrow$ authenticated public key

Step 2: Establishing a Shared Secret

Question

Once Alice has an authenticated public key of Bob, how can they establish a common secret?

There are several approaches:

- Key-exchange protocols, such as Diffie–Hellman.
- Public-key encryption based key establishment.
- Other cryptographic key-establishment mechanisms.

Traditionally, many key-exchange protocols were based on the **Discrete Logarithm Problem (DLP)**.

Alice $\longleftrightarrow$ Bob

K_{AB}

Both parties should obtain the same secret K_{AB} without revealing it to an adversary.

Question: Why Not Simply Use PKE?

Suppose Bob publishes (pk_B, sk_B) .

Alice chooses a random secret K and sends

$$c = \text{Enc}_{pk_B}(K).$$

Bob computes

$$K = \text{Dec}_{sk_B}(c).$$

Question

Does this immediately give us a good key-establishment mechanism?

Question: Why Not Simply Use PKE?

Suppose Bob publishes (pk_B, sk_B) .

Alice chooses a random secret K and sends

$$c = \text{Enc}_{pk_B}(K).$$

Bob computes

$$K = \text{Dec}_{sk_B}(c).$$

Question

Does this immediately give us a good key-establishment mechanism?

Not necessarily. We must consider stronger security properties, including:

- protection against active attacks,
- chosen-ciphertext security,
- authentication,
- **forward secrecy**,
- key compromise and compromise recovery.

Key question

Can we design a post-quantum key-establishment mechanism with strong security properties?

Step 3: Secure Symmetric Communication

Once Alice and Bob establish a shared secret K_{AB} , they can use symmetric cryptography for the actual communication.

Question

Why do we not use public-key cryptography to encrypt every message?

Step 3: Secure Symmetric Communication

Once Alice and Bob establish a shared secret K_{AB} , they can use symmetric cryptography for the actual communication.

Question

Why do we not use public-key cryptography to encrypt every message?

Symmetric encryption is much more efficient for large amounts of data.

$$C_i = \text{Enc}_{K_{AB}}(M_i)$$

$$M_i = \text{Dec}_{K_{AB}}(C_i).$$

We assume that the block-cipher and stream-cipher constructions used here provide the required post-quantum security.

Public-key cryptography establishes K_{AB}

Symmetric cryptography protects the data

Where Are We in the Post-Quantum Setting?

What have we already discussed?

We have discussed **hash-based digital signatures** as an example of a post-quantum signature mechanism.

Therefore, the authentication component can be constructed using post-quantum signatures:

$$\text{CA} \xrightarrow{\text{PQ signature}} \text{Certificate}$$

and users can verify authenticated public keys.

Assumption

For this lecture, we will not discuss the construction of block ciphers or stream ciphers. We assume that the symmetric primitives provide the required post-quantum security.

Our focus: Post-Quantum Key Establishment

Step 2: Key Establishment is Our Focus

There are multiple approaches to establishing a shared secret:

- 1 Diffie–Hellman type key exchange,
- 2 lattice-based mechanisms,
- 3 code-based mechanisms,
- 4 hash-based mechanisms,
- 5 isogeny-based mechanisms,
- 6 other post-quantum approaches.

Question

Today, can we explore one completely different mathematical approach to key establishment?

Step 2: Key Establishment is Our Focus

There are multiple approaches to establishing a shared secret:

- 1 Diffie–Hellman type key exchange,
- 2 lattice-based mechanisms,
- 3 code-based mechanisms,
- 4 hash-based mechanisms,
- 5 isogeny-based mechanisms,
- 6 other post-quantum approaches.

Question

Today, can we explore one completely different mathematical approach to key establishment?

Code-Based Cryptography

Coding Theory

Error Detecting code

Why Do We Need Coding Theory?

Question

Suppose Alice transmits a message to Bob over a communication channel. Is the transmitted message always received correctly?

Why Do We Need Coding Theory?

Question

Suppose Alice transmits a message to Bob over a communication channel. Is the transmitted message always received correctly?

Not necessarily. During transmission, bits can be altered because of

- physical noise in the communication channel,
- electromagnetic interference,
- hardware imperfections,
- adverse weather or physical conditions,
- other non-adversarial transmission errors.

Question

Can we add some redundancy to the transmitted message so that Bob can detect or correct such errors?

Coding Theory: Add structured redundancy to protect information

A Simple Example: 2-Bit Messages

Consider the message space

$$\mathcal{M} = \{00, 01, 10, 11\}.$$

Thus, a message is

$$m = (m_1, m_2) \in \mathbb{F}_2^2.$$

Question

How can we encode these four possible messages so that Bob can detect transmission errors?

Let us first try a very simple idea:

$$(m_1, m_2) \longrightarrow (m_1, m_2, m_1, m_2)$$

We simply repeat every bit once.

Examples:

$$00 \rightarrow 0000, \quad 01 \rightarrow 0101, \quad 10 \rightarrow 1010, \quad 11 \rightarrow 1111.$$

Question

What types of bit errors can this encoding detect or correct?

Repetition Code: Can We Detect an Error?

The encoding is

$$(m_1, m_2) \longrightarrow (m_1, m_2, m_1, m_2).$$

Suppose one bit is flipped.

For example,

$$01 \longrightarrow 0101 \longrightarrow 0111.$$

The received word is no longer of the form

$$(m_1, m_2, m_1, m_2).$$

Therefore, Bob can detect that an error has occurred.

Observation

Any single-bit flip produces a word outside the set of valid codewords.

Hence,

Any 1-bit error can be detected.

Question

What happens if two bits are flipped?

Repetition Code: What If Two Bits Flip?

Consider

$$01 \longrightarrow 0101.$$

Flip the first and third bits:

$$0101 \longrightarrow 1111.$$

But

$$1111$$

is itself a valid encoding of the message 11.

Therefore Bob cannot distinguish between

01 with two errors

and

11 with no errors.

Conclusion

The repetition code

$$(m_1, m_2) \mapsto (m_1, m_2, m_1, m_2)$$

Why Can We Not Correct a 1-Bit Error?

Consider the received word

0111.

It could have originated from

0101

by flipping the third bit.

But it could also have originated from

1111

by flipping the first bit.

Thus Bob cannot determine uniquely which message was transmitted.

Key distinction

Detection $\neq$ Correction

Detection asks: "Did an error occur?"

Correction asks: "Which valid codeword was actually transmitted?"

Can We Reduce the Redundancy?

The repetition encoding maps

2 bits $\longrightarrow$ 4 bits.

Thus the message expansion is

$$\frac{4 - 2}{2} = 100\%.$$

Question

Do we really need to repeat every bit?

Can we introduce less redundancy while still detecting every 1-bit error?

Can We Reduce the Redundancy?

The repetition encoding maps

2 bits $\longrightarrow$ 4 bits.

Thus the message expansion is

$$\frac{4 - 2}{2} = 100\%.$$

Question

Do we really need to repeat every bit?

Can we introduce less redundancy while still detecting every 1-bit error?

Consider the encoding

$$(m_1, m_2) \longrightarrow (m_1, m_2, m_1 \oplus m_2).$$

Examples:

$$\begin{array}{ll} 00 \rightarrow 000, & 01 \rightarrow 011, \\ 10 \rightarrow 101, & 11 \rightarrow 110. \end{array}$$

Now we have

2 bits $\longrightarrow$ 3 bits.

The message expansion is only

50%

Parity Encoding: Can We Detect One Error?

The valid codewords are

$$\mathcal{C} = \{000, 011, 101, 110\}.$$

Notice that every valid codeword has **even parity**.

Question

What happens when exactly one bit is flipped?

A single bit flip changes the parity from even to odd.

For example,

$$011 \longrightarrow 001.$$

But

$$001 \notin \mathcal{C}.$$

Therefore Bob can detect the error.

Conclusion

The parity encoding can detect every single-bit error.

1-bit error $\Rightarrow$ invalid codeword
--

But Can We Correct One Bit?

Consider the received word

001.

It is not a valid codeword.

Question

Which bit was flipped?

There are three possibilities:

$001 \xrightarrow{\text{flip bit 1}} 101,$

$001 \xrightarrow{\text{flip bit 2}} 011,$

$001 \xrightarrow{\text{flip bit 3}} 000.$

All three are valid codewords.

Therefore, detecting the error does not tell us which bit was corrupted.

Conclusion

The parity code $(m_1, m_2) \mapsto (m_1, m_2, m_1 \oplus m_2)$

- detects every 1-bit error,
- does not uniquely correct a 1-bit error.

From Encoding Rules to Codes

We have now seen two encoding mechanisms.

Message	Encoding	Length
(m_1, m_2)	(m_1, m_2, m_1, m_2)	4
(m_1, m_2)	$(m_1, m_2, m_1 \oplus m_2)$	3

In both cases, only some strings are used as valid encoded messages.

Question

Why not simply choose some subset of $\{0, 1\}^n$ as our valid encoded messages?

This motivates the notion of a **code**:

$$\mathcal{C} \subseteq \mathbb{F}_2^n$$

where the elements of $\mathcal{C}$ are called **codewords**.



Why Do We Want Algebraic Structure?

We could choose a random mapping

$$\mathcal{M} \longrightarrow \mathcal{C}.$$

But a random collection of codewords gives us little mathematical structure.

Question

Can we choose the codewords so that they satisfy useful algebraic properties?

One important example is a **linear code**.

A linear code satisfies

$$c_1, c_2 \in \mathcal{C} \implies c_1 + c_2 \in \mathcal{C}.$$

Also,

$$0 \in \mathcal{C}.$$

Thus the set of valid codewords forms a vector space over $\mathbb{F}_2$.

Linear Code = Code with vector-space structure

Generating Matrix: Encoding as a Linear Transformation

For a linear code, encoding can be represented as a linear transformation.

For a message

$$m \in \mathbb{F}_2^k,$$

we can write

$$c = mG$$

where

$$G \in \mathbb{F}_2^{k \times n}$$

is called the **generator matrix**.

Why is it called a generator matrix?

The rows of G generate all possible codewords through linear combinations.

Thus,

$$\mathcal{C} = \{mG : m \in \mathbb{F}_2^k\}.$$

$$\text{Message} \xrightarrow{G} \text{Codeword}$$

Generating Matrix for the Repetition Code

Our first encoding was

$$(m_1, m_2) \longrightarrow (m_1, m_2, m_1, m_2).$$

We can write

$$(m_1, m_2) \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} = (m_1, m_2, m_1, m_2).$$

Therefore, the generator matrix is

$$G = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}.$$

Question

What does each row of G represent?

The first row is the encoding of 10:

$$10 \rightarrow 1010.$$

The second row is the encoding of 01:

$$01 \rightarrow 0101.$$

Generating Matrix for the Parity Code

Our second encoding was

$$(m_1, m_2) \longrightarrow (m_1, m_2, m_1 \oplus m_2).$$

Since addition in $\mathbb{F}_2$ is XOR,

$$m_1 \oplus m_2 = m_1 + m_2.$$

Therefore,

$$(m_1, m_2) \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} = (m_1, m_2, m_1 + m_2).$$

Hence,

$$G = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}.$$

Key observation

Once we have G , encoding is simply matrix multiplication:

$$c = mG.$$

Why Is the Generator Matrix Important?

For a linear code,

$$c = mG.$$

Therefore, the generator matrix completely specifies the encoding transformation.

It determines:

- how messages are mapped to codewords,
- the set of valid codewords,
- the redundancy introduced by the code,
- important structural properties of the code.

Question

If G determines the structure of a linear code, what happens if we publish G ?

Why Is the Generator Matrix Important?

For a linear code,

$$c = mG.$$

Therefore, the generator matrix completely specifies the encoding transformation.

It determines:

- how messages are mapped to codewords,
- the set of valid codewords,
- the redundancy introduced by the code,
- important structural properties of the code.

Question

If G determines the structure of a linear code, what happens if we publish G ?

For ordinary error correction, this is perfectly fine.

But for cryptography

What if we can construct a code for which decoding is easy with some secret information, but hard when only its public representation is known?



Coding Theory

Error Correcting code

That Detects and corrects errors

September 12, 2026

From Error Detection to Error Correction

Question

So far, we have constructed codes that can **detect** a single-bit error. Can we construct a code that can also **correct** the error?

A simple idea is to repeat every bit three times:

$$(m_1, m_2) \longrightarrow (m_1, m_1, m_1, m_2, m_2, m_2).$$

For example,

$$\begin{aligned} 00 &\rightarrow 000000, & 01 &\rightarrow 000111, \\ 10 &\rightarrow 111000, & 11 &\rightarrow 111111. \end{aligned}$$

Question

If one of the transmitted bits is flipped, can we recover the original message?

Repeat each bit three times

Three-Repetition Code: Error Detection

Consider the encoding

$$(m_1, m_2) \longrightarrow (m_1, m_1, m_1, m_2, m_2, m_2).$$

The valid codewords are

$$\mathcal{C} = \{000000, 000111, 111000, 111111\}.$$

Suppose one bit is flipped:

$$000111 \longrightarrow 010111.$$

The received word is not one of the valid codewords.

Therefore,

Every 1-bit error can be detected.

Question

Can we do more than just detect the error?

Can we determine which bit was flipped and recover the original codeword?

Three-Repetition Code: Error Correction

Suppose

000111

was transmitted and the second bit is flipped:

000111 $\rightarrow$ 010111.

Look at each group of three bits:

010 111.

The first group is

010.

Two of the three bits are 0, so we infer that the original bit was 0.

This is the **majority decoding rule**:

000, 001, 010, 100 $\rightarrow$ 0

111, 110, 101, 011 $\rightarrow$ 1.

Hence

010111 $\xrightarrow{\text{majority decoding}}$ 000111 $\rightarrow$ 01.

Conclusion

The code can detect and correct every single-bit error.

Question: What If Two Bits Are Flipped?

Suppose 000111 is transmitted.

Now flip two bits in the first group: $000111 \rightarrow 110111$.

The first group becomes

110.

Majority decoding gives

$110 \rightarrow 1$.

Thus Bob decodes

$110111 \rightarrow 111111$,

which corresponds to the message

11

instead of the original

01.

Therefore,

Two errors cannot be guaranteed to be detected or corrected.

Important observation

The code guarantees correction only when the number of errors is at most one.

Generating Matrix of the Three-Repetition Code

The encoding is

$$(m_1, m_2) \longrightarrow (m_1, m_1, m_1, m_2, m_2, m_2).$$

Using

$$c = mG,$$

we obtain

$$G = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \end{bmatrix}.$$

Indeed,

$$(m_1, m_2) \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \end{bmatrix} = (m_1, m_1, m_1, m_2, m_2, m_2).$$

Question

What does each row of G represent?

The rows are the encoded versions of the basis messages:

$$10 \rightarrow 111000, \quad 01 \rightarrow 000111.$$

$$c = mG$$

Decoding Algorithm: Majority Decoding

Let the received word be

$$r = (r_1, r_2, r_3, r_4, r_5, r_6).$$

Separate it into two groups:

$$(r_1, r_2, r_3) \quad (r_4, r_5, r_6).$$

Decoding

For each group, use majority voting:

$$\hat{m}_1 = \text{Maj}(r_1, r_2, r_3),$$

$$\hat{m}_2 = \text{Maj}(r_4, r_5, r_6).$$

The decoded message is $\hat{m} = (\hat{m}_1, \hat{m}_2)$.

For example, $r = 010101$.

Then $010 \rightarrow 0$, $101 \rightarrow 1$.

Therefore $\hat{m} = 01$.

Guarantee

The algorithm correctly recovers the message whenever at most one bit has been flipped.

The Cost of Error Correction

Our original message contains

2 bits.

The encoded message contains

6 bits.

Therefore the additional redundancy is

$$\frac{6 - 2}{2} \times 100\% = 200\%.$$

Question

We can now correct one-bit errors, but do we really want to transmit three times as many bits?

2 message bits $\longrightarrow$ 6 transmitted bits

Error correction is powerful, but redundancy is expensive.

Can We Reduce the Redundancy?

We have seen three different constructions:

Code	Length	Detect 1 error?	Correct 1 error?
Repetition	4	✓	×
Parity	3	✓	×
3-Repetition	6	✓	✓

Question

Can we design a code that

- corrects errors,
- uses much less redundancy,
- has an efficient encoding algorithm, and
- has an efficient decoding algorithm?

Can we get better error correction with less redundancy?

Some Important Error-Correcting Codes

There are many families of error-correcting codes designed for different error models and applications.

- **Hamming codes:** efficient correction of a small number of errors.
- **Reed–Solomon codes:** widely used for burst and symbol errors.
- **BCH codes:** powerful algebraic error-correcting codes.
- **Reed–Muller codes:** structured linear codes with efficient decoding.
- **LDPC codes:** sparse parity-check matrices and efficient iterative decoding.
- **Polar codes:** capacity-achieving codes with efficient encoding and decoding.
- **Goppa codes:** important in code-based cryptography.

Error Correcting Code

to

Code-based Public Key Encryption

An Overview

From Error Correction to Code-Based Cryptography

For communication, we ask:

Can Bob decode a noisy codeword efficiently?

For cryptography, we ask a different question:

Cryptographic Question

Can Bob decode efficiently using some secret information, while an attacker cannot efficiently decode using only the public information?

This suggests a trapdoor structure:

Easy decoding with secret information

Hard decoding without secret information

Error-Correcting Code $\longrightarrow$ Decoding Problem $\longrightarrow$ Code-Based Cryptography

Next

We will see how this idea leads to a public-key encryption scheme.

From Error-Correcting Codes to Public-Key Encryption

Recall: Error-Correcting Codes

- A linear error-correcting code has a **generator matrix** G .
- G maps a message m to a codeword c : $c = mG$
- Thus, Message Space $\xrightarrow{G}$ Code Space.
- If a codeword is corrupted by a sufficiently small number of errors, the error-correcting algorithm can recover the original codeword.

$$m \xrightarrow{G} c \xrightarrow{+e} y = c + e$$

where e is an error vector with weight below the correction capability.

The Basic Idea of Code-Based PKE

Key observation:

What if we deliberately introduce errors during encryption, but only the legitimate receiver can efficiently correct them?

Start with an error-correcting code having generator matrix G .

- In coding theory, G is normally public.
- If G and its efficient decoder are public, anyone can decode.
- Therefore, we need to **hide the special structure of the code**.

Hide the structure of G + Add correctable errors

This gives the fundamental idea behind code-based PKE.

Hiding the Structure of the Code

Let

$$G \in \mathbb{F}_2^{k \times n}$$

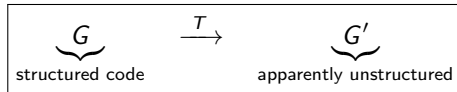
be the generator matrix of an error-correcting code.

We apply a secret transformation

$$G' = T(G)$$

where T hides the special structure of the original code.

- G' is published as the **public key**.
- The information needed to undo the transformation and exploit the special decoding algorithm is kept as the **secret key**.



The transformation T should preserve the ability to encode while hiding the efficient decoding structure.

Encryption

The sender knows only the public generator matrix

$$G' = T(G).$$

Given a message

$$m \in \mathbb{F}_2^k,$$

the sender first computes

$$c = mG'.$$

Then deliberately introduces an error vector e :

$$y = c + e = mG' + e.$$

The error satisfies

$$\text{wt}(e) \leq t,$$

where t is the error-correction capability of the underlying code.

Therefore,

$$\text{Ciphertext } y = mG' + e.$$

$$m \xrightarrow{G'} c \xrightarrow{+e} y$$

Decryption

The receiver knows the secret information associated with T and the underlying error-correcting code. Given

$$y = mG' + e,$$

the receiver first uses the secret decoding capability to remove the error:

$$y \longrightarrow c = mG'.$$

Then the receiver uses the secret transformation information to recover the original message:

$$mG' \xrightarrow{T^{-1}} m.$$

Thus,

$$y \xrightarrow{\text{secret decoding}} mG' \xrightarrow{\text{secret transformation}} m$$

Only the legitimate receiver can efficiently perform the complete decryption process.

Where is the Security?

The security does **not** come simply from adding errors.

The security comes from the difference between:

Efficient decoding with the hidden structure

and

Hard decoding without the hidden structure.

The attacker knows

$$G', \quad y = mG' + e,$$

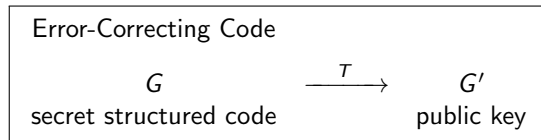
but does not know the secret structure.

Therefore, the attacker faces a hard decoding problem:

Given G' and y , recover m or e .

For code-based cryptography, this is closely related to the **syndrome decoding problem**.

The Fundamental Construction



Encryption:

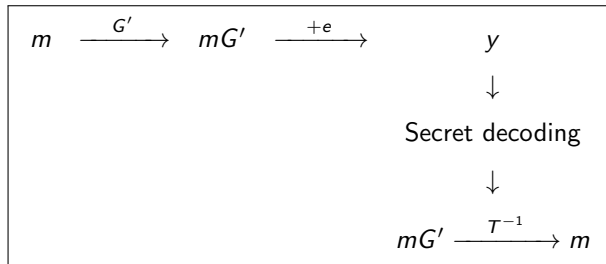
$$y = mG' + e$$

Decryption:

$$y \xrightarrow{\text{secret decoder}} mG' \xrightarrow{\text{secret transformation}} m$$

Public encoding + deliberate errors + hidden efficient decoding

The Big Picture



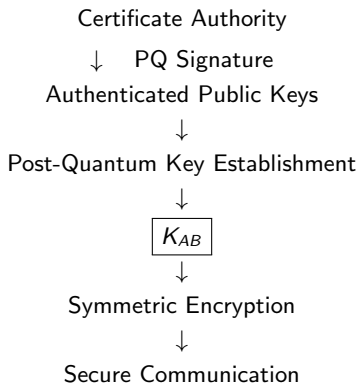
Core principle:

Public: efficient encoding
Secret: efficient error correction
Attacker: hard decoding problem

This is the fundamental idea behind **code-based public-key encryption**.

Reason behind studying Code-based PKE

Cryptographically Secure Channel



Broader system view

Once we have a secure PKI and a secure post-quantum key-establishment mechanism, a client and server can establish a secure channel and then communicate securely, including while executing privacy-sensitive AI/ML algorithms.

Why Study Code-Based PKE?

Recall: Building a Cryptographically Secure Channel

- In PKI, a user's public key is authenticated by a **Certificate Authority (CA)** using a digital signature.
- We have studied **hash-based digital signatures**, which can provide post-quantum secure authentication of public keys.
- Once the public key is authenticated, two parties need a **post-quantum secure mechanism to establish a shared secret**.

A public-key encryption scheme (PKE) or key encapsulation mechanism (KEM) can be used for this purpose.

Authenticated Public Key + PQ-secure PKE/KEM $\longrightarrow$ Shared Secret
--

The shared secret is then used as a session key, typically through a **KDF** and symmetric encryption

Why Code-Based PKE?

Code-based cryptography provides another approach to post-quantum public-key encryption.

- It is based on the hardness of decoding certain error-correcting codes.
- The underlying error-correcting code provides an efficient decoding trapdoor for the legitimate receiver.
- Without the secret structure, decoding is believed to be computationally hard.

Thus, a code-based PKE/KEM can be used to establish a shared secret:

Code-Based PKE/KEM	→	Shared Secret	→	KDF	→	Session Key
--------------------	---	---------------	---	-----	---	-------------

So, how do we establish a shared secret using code-based PKE?

PKE vs. KEM

Public-Key Encryption (PKE)

$$\text{Enc}(pk, m) \rightarrow c$$

$$\text{Dec}(sk, c) \rightarrow m$$

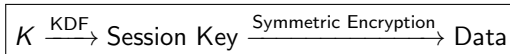
- Designed to encrypt a message m .
- The receiver recovers the original message.

Key Encapsulation Mechanism (KEM)

$$\text{Encaps}(pk) \rightarrow (c, K)$$

$$\text{Decaps}(sk, c) \rightarrow K$$

- Designed to establish a fresh shared secret K .
- The secret K is not sent directly.
- The ciphertext c allows the receiver to recover the same K .



From PKE to Key Establishment

The basic idea can be viewed as:

Choose random secret r $\longrightarrow$ Public-Key Encryption $\longrightarrow c$

The receiver uses the secret key to recover r .

$c \xrightarrow{\text{Decaps}} r \xrightarrow{\text{KDF}} K$

Thus, both parties obtain the same session key:

$K_{\text{Alice}} = K_{\text{Bob}}$

Important distinction:

- PKE encrypts a message.
- KEM establishes a shared secret.

Code-Based KEM $\longrightarrow$ Shared Secret $\longrightarrow$ Session Key

KEM: From Encryption to Shared Secret

A KEM can be viewed conceptually as:

Choose a fresh random value r , then $r \xrightarrow{\text{PKE Encryption}} C$

The receiver uses the secret key to recover r : $C \xrightarrow{\text{PKE Decryption}} r$

Both parties then derive the shared secret: $K = \text{KDF}(r)$

Thus, $\text{Random } r \rightarrow \text{Ciphertext } C \rightarrow \text{Shared Secret } K$

Security requirement:

For modern cryptographic use, the underlying construction must provide strong security against **chosen-ciphertext attacks (CCA)**.

CCA-secure construction $\implies$ secure key encapsulation

The CCA requirement prevents an attacker from exploiting maliciously modified ciphertexts through the decapsulation process.

Textbook:

- 1 Daniel J. Bernstein, Johannes Buchmann, and Erik Dahmen. *Post-Quantum Cryptography*. Springer, 2009. ISBN: 9783540887010.

Acknowledgment:

- Some figures in this work were generated with the assistance of ChatGPT (OpenAI). It is used to decorate contents as well.



Dr. Laltu Sardar, Assistant Professor, IISER Thiruvananthapuram
laltu.sardar@iisertvm.ac.in, laltu.sardar.crypto@gmail.com
Course webpage: https://laltu-sardar.github.io/courses/26_crypto.html.